Tugas Besar 1
Kelompok G04 – BUMC

IF3170 Inteligensi Artifisial

Pencarian Solusi Pengisian Peti Kemas Truk
dengan Algoritma *Local Search*

Semester I Tahun 2026/2027



Disusun oleh:

Kelompok G04 – BUMC

Raynard Fausta	(13524052)
Jason Edward Salim	(13524034)
Athilla Zaidan Zidna Fann	(13524068)

Laboratorium Artificial Intelligence
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Persoalan **5**
1.1 Latar Belakang . . . 5
1.2 Variabel Persoalan . . . 5
1.3 Fungsi Objektif dan Batasan . . . 6
1.4 Instans Uji . . . 6

2 Dasar Teori **7**
2.1 Local Search . . . 7
2.2 Hill-Climbing . . . 7
2.3 Simulated Annealing . . . 7
2.4 Genetic Algorithm . . . 8

3 Pembahasan: Implementasi **9**
3.1 Struktur Program . . . 9
3.2 Representasi State . . . 10
3.3 Batasan dan Fungsi Objektif . . . 11
3.4 Heuristik Repair . . . 12
3.5 Aksi Tetangga . . . 13
3.6 State Awal . . . 15
3.7 Batas Atas Optimum Global . . . 16
3.8 Hill-Climbing . . . 16
3.9 Simulated Annealing . . . 18
3.10 Genetic Algorithm . . . 20
3.11 Bonus: Fungsi Objektif Berbasis ETA . . . 22
3.12 Bonus: Multi-Truk . . . 22
3.13 Visualisasi dan Keluaran . . . 23

4 Pembahasan: Hasil dan Analisis Eksperimen **24**
4.1 Metodologi Eksperimen . . . 24
4.2 Hasil Hill-Climbing . . . 24
4.3 Hasil Simulated Annealing . . . 26
4.4 Hasil Genetic Algorithm . . . 29
4.5 Hasil Bonus: Objektif ETA . . . 30
4.6 Hasil Bonus: Multi-Truk . . . 31
4.7 Analisis . . . 33
4.7.1 Seberapa dekat dengan optimum global? . . . 33
4.7.2 Perbandingan antaralgoritma . . . 33

4.7.3 Perbandingan durasi 34
4.7.4 Konsistensi hasil 34
4.7.5 Pengaruh parameter Simulated Annealing 35
4.7.6 Pengaruh populasi dan generasi pada Genetic Algorithm . . . 35

5 Kesimpulan dan Saran **36**
5.1 Kesimpulan 36
5.2 Saran 37

6 Pembagian Tugas **38**

A Lampiran **40**
A.1 Tautan Repository 40
A.2 Hasil Setiap Run Genetic Algorithm 40
A.3 Pernyataan Penggunaan AI Generatif 41

Daftar Tabel

1.1 Atribut paket dan truk 5
1.2 Instans uji 6

3.1 Modul program 9

4.1 Ringkasan Hill-Climbing (rata-rata 3 *run*) 25
4.2 Hasil setiap *run* Hill-Climbing 25
4.3 Hasil setiap *run* Simulated Annealing (geometrik, 20.000 iterasi, $p_0 =$ 0,01, $T_{\min} = 0{,}05$) 27
4.4 Pengaruh parameter Simulated Annealing pada instans *medium* (rata-rata 3 *run*) 28
4.5 Pengaruh *ratio bias* pada instans *large* (rata-rata 3 *run*) 29
4.6 Ringkasan Genetic Algorithm (rata-rata 3 *run*) 30
4.7 Perbandingan objektif *value* dan ETA (SA, instans *medium*, *seed* 1) . 31
4.8 Hasil multi-truk dibanding satu truk (rata-rata 3 *run*) 31
4.9 Perbandingan konfigurasi terbaik tiap algoritma (rata-rata 3 *run*) . . 33

6.1 Pembagian tugas anggota kelompok 38

A.1 Hasil setiap *run* Genetic Algorithm 40
A.2 Penggunaan AI generatif 41

Daftar Gambar

3.1 Visualisasi *state* awal dan akhir (HC *steepest-ascent*, instans *medium*, *seed* 1) 23

4.1 Nilai objektif terhadap iterasi Hill-Climbing (instans *medium*, *seed* 1) 25
4.2 Sebaran nilai akhir varian Hill-Climbing (instans *large*) 25
4.3 Simulated Annealing pada instans *medium* (*seed* 1) 27
4.4 Perbandingan konfigurasi Simulated Annealing (instans *medium*) . . 29
4.5 Nilai objektif maksimum GA terhadap generasi (instans *large*; garis = rata-rata 3 *run*, area = rentang) 30
4.6 GA populasi 100, generasi 100 (instans *large*, *seed* 1) 30
4.7 *State* akhir SA dengan dua truk (instans *large*, *seed* 1) 32

Bab 1

Deskripsi Persoalan

1.1 Latar Belakang

Mejiro Express ingin menguji coba layanan logistik darat dengan satu truk peti kemas. Setiap pengiriman perlu mengatur paket mana yang dimuat, di mana paket diletakkan, dan bagaimana paket diputar agar total nilai (*value*) paket yang terangkut sebesar mungkin, tanpa melanggar batasan fisik peti kemas. Persoalan ini merupakan varian persoalan *3D knapsack / 3D bin packing* yang tergolong NP-hard, sehingga pencarian solusi eksak tidak praktis untuk ukuran instans yang realistis. Tugas ini menyelesaikannya dengan algoritma *local search*: Hill-Climbing, Simulated Annealing, dan Genetic Algorithm [1].

1.2 Variabel Persoalan

Setiap paket berbentuk balok dengan atribut berikut.

Tabel 1.1: Atribut paket dan truk

Entitas	**Atribut**	**Keterangan**
Paket	`id`	Identitas paket
	`w, l, h`	Lebar, panjang, dan tinggi paket
	`orientation`	Sumbu truk yang sejajar dengan sisi `w`, `l`, `h`; ada enam kemungkinan
	`value`	Nilai paket
	`weight`	Berat paket
	`fragile`	Paket pecah-belah atau tidak
	`eta`	Perkiraan sisa hari hingga paket harus tiba
Truk	`w, l, h`	Dimensi peti kemas
	`max_capacity`	Berat maksimum yang dapat diangkut

1.3 Fungsi Objektif dan Batasan

Fungsi objektif yang dimaksimalkan adalah total *value* paket yang berada di dalam peti kemas:

$$f(s) = \sum_{i\,:\,\text{paket } i \text{ di dalam truk}} \text{value}_i. \tag{1.1}$$

Paket yang berada di luar truk tidak menyumbang nilai. Sebuah konfigurasi dianggap valid apabila memenuhi seluruh batasan berikut.

1. **Tidak menembus dinding.** Seluruh bagian paket di dalam truk berada di dalam peti kemas.
2. **Tidak saling bertumpuk ruang.** Dua paket tidak boleh menempati ruang yang sama.
3. **Tidak melayang.** Paket dengan sudut kiri-bawah-belakang (x_i, y_i, z_i) dan alas $B_i = \{(x, y, z_i) \mid x_i \le x < x_i + w_i,\ y_i \le y < y_i + d_i\}$ harus memiliki minimal satu titik alas yang ditopang, yaitu $z_i = 0$ (lantai truk) atau tepat berada di atas permukaan atas paket lain.
4. **Barang pecah-belah tidak menjadi penopang.** Tidak boleh ada paket yang bertumpu di atas paket *fragile*.
5. **Kapasitas.** Total berat paket di dalam truk tidak melebihi `max_capacity`.

Koordinat berupa bilangan bulat. State awal dibangkitkan secara acak. Aksi tetangga yang diperbolehkan adalah (1) menukar dua paket, baik di dalam maupun di luar truk, (2) memindahkan paket ke koordinat lain, dan (3) merotasi paket 90° terhadap sumbu x, y, atau z.

1.4 Instans Uji

Tiga instans dibangkitkan dengan `src/generate_instance.py` (*seed* 42) dan disimpan di `data/instances/`. Kolom "batas atas" adalah nilai optimum *knapsack* 0/1 dengan batasan berat dan volume yang mengabaikan geometri (Subbab 3.7); nilai ini tidak lebih kecil daripada optimum global persoalan.

Tabel 1.2: Instans uji

Instans	**Truk** $(w \times l \times h)$	**Kapasitas**	**Jumlah paket**	**Batas atas**
small	$6 \times 8 \times 5$	150	15	734
medium	$10 \times 20 \times 5$	500	55	2384
large	$12 \times 24 \times 8$	1200	120	5633

Bab 2

Dasar Teori

2.1 Local Search

Local search bekerja pada satu (atau sekumpulan) *state* lengkap dan berpindah ke *state* tetangga untuk memperbaiki nilai fungsi objektif. Jalur pencarian tidak disimpan, sehingga kebutuhan memorinya kecil dan cocok untuk persoalan optimasi berukuran besar seperti persoalan pengisian peti kemas ini. Kelemahannya, pencarian dapat terjebak di *local optimum*, *plateau*, atau *ridge* [2, 3].

2.2 Hill-Climbing

Hill-Climbing selalu berpindah ke tetangga yang lebih baik dan berhenti ketika tidak ada tetangga yang lebih baik [2, 4]. Varian yang diimplementasikan:

- **Steepest-ascent**: membangkitkan semua tetangga dan memilih yang terbaik. Berhenti bila tetangga terbaik tidak lebih baik daripada *state* sekarang.
- **Stochastic**: memilih secara acak salah satu tetangga yang lebih baik.
- **Sideways move**: seperti *steepest-ascent*, tetapi boleh berpindah ke tetangga yang bernilai sama untuk keluar dari *plateau*, dibatasi oleh parameter *maximum sideways move*.
- **Random-restart**: menjalankan *steepest-ascent* berulang kali dari *state* awal acak yang berbeda dan mengambil hasil terbaik, dibatasi oleh parameter *maximum restart*.

2.3 Simulated Annealing

Simulated Annealing [5, 6] memilih satu tetangga acak setiap iterasi. Tetangga yang lebih baik ($\Delta E > 0$) selalu diterima, sedangkan tetangga yang lebih buruk diterima dengan peluang $e^{\Delta E/T}$. Suhu T diturunkan bertahap sesuai jadwal (*schedule*). Pada suhu tinggi pencarian bersifat eksploratif, sedangkan pada suhu rendah perilakunya mendekati Hill-Climbing. Dengan demikian SA dapat keluar dari *local optimum* di awal pencarian.

2.4 Genetic Algorithm

Genetic Algorithm [2, 7, 8] memelihara populasi berisi k *state* (individu). Nilai setiap individu diukur dengan *fitness function*. Generasi berikutnya dibentuk melalui tiga operator:

1. **Seleksi**: induk dipilih dengan peluang sebanding *fitness* (*roulette wheel*).
2. **Crossover**: dua induk dipotong pada titik acak yang sama, lalu bagian ekornya ditukar sehingga terbentuk dua anak.
3. **Mutasi**: anak diubah secara acak dengan peluang kecil.

Bab 3

Pembahasan: Implementasi

Seluruh program ditulis dalam Python dan berada di folder `src/`. Bab ini menjelaskan representasi persoalan, heuristik yang dipakai bersama oleh semua algoritma, lalu implementasi setiap algoritma. Potongan kode diambil langsung dari *repository*.

3.1 Struktur Program

Tabel 3.1: Modul program

Modul	Isi
`core/models.py`	Data statis: `Package`, `Truck`, `Problem`
`core/validation.py`	Validasi tipe dan rentang parameter serta data instans
`core/state.py`	Representasi *state*: `Placement` dan `State`
`core/orientation.py`	Enam orientasi paket dan rotasi 90°
`core/geometry.py`	Operasi balok: tumpang tindih, tumpuan, dinding
`core/constraints.py`	Pendeteksian pelanggaran batasan
`core/objective.py`	Fungsi objektif *value* dan ETA
`core/repair.py`	Heuristik *repair state* menjadi valid
`core/neighbors.py`	Aksi *swap*, *move*, *rotate* dan pembangkit tetangga
`core/initial.py`	*State* awal acak
`core/bounds.py`	Batas atas optimum global (*knapsack*)
`core/io.py`, `core/result.py`	Baca/tulis instans dan hasil *run* (JSON)
`algorithms/`	Hill-Climbing, Simulated Annealing, Genetic Algorithm
`experiments/`	Skrip eksperimen HC, SA, dan GA
`viz/`	Visualisasi 3D *state* dan plot hasil eksperimen
`main.py`	Antarmuka baris perintah untuk satu kali *run*

3.2 Representasi State

Data paket dan truk tidak berubah selama pencarian, sehingga dipisahkan dari *state*. Sebuah *state* adalah *tuple* berisi satu `Placement` untuk setiap paket, berurutan sesuai indeks paket. `Placement` menyimpan apakah paket berada di dalam truk, koordinat sudut kiri-bawah-belakang (x, y, z), dan orientasinya. Orientasi ditulis seperti pada spesifikasi, yaitu sumbu truk yang sejajar dengan sisi `w`, `l`, dan `h`; misalnya `("y","x","z")` berarti {w:y, l:x, h:z}. Karena semua objek bersifat *immutable* (`frozen=True`), *state* dapat langsung dibandingkan dan disimpan di dalam *set*.

Kode 3.1: Model persoalan (`src/core/models.py`)

```python
@dataclass(frozen=True)
class Package:
    id: str
    w: int
    l: int
    h: int
    value: int
    weight: int
    fragile: bool
    eta: int

    def __post_init__(self):
        if not isinstance(self.id, str) or not self.id.strip():
            raise ValueError("package id must be a nonempty string")
        for name in ("w", "l", "h", "weight"):
            require_integer(getattr(self, name), f"package {self.id}: {name}", 1)
        require_number(self.value, f"package {self.id}: value")
        require_number(self.eta, f"package {self.id}: eta")
        if self.eta == 0:
            raise ValueError(f"package {self.id}: eta must be > 0")
        if type(self.fragile) is not bool:
            raise ValueError(f"package {self.id}: fragile must be a boolean")


@dataclass(frozen=True)
class Truck:
    w: int
    l: int
    h: int
    max_capacity: int

    def __post_init__(self):
        for name in ("w", "l", "h", "max_capacity"):
            require_integer(getattr(self, name), f"truck: {name}", 1)


@dataclass(frozen=True)
class Problem:
    truck: Truck
    packages: tuple[Package, ...]
    objective: str = "value"  # "value" | "eta"
    truck_count: int = 1

    def __post_init__(self):
        require_integer(self.truck_count, "truck_count", 1)
        if self.objective not in ("value", "eta"):
            raise ValueError("objective must be value or eta")
        ids = [p.id for p in self.packages]
        if len(set(ids)) != len(ids):
            raise ValueError("duplicate package id")
```

Kode 3.2: Representasi *state* (`src/core/state.py`)

```python
from dataclasses import dataclass, replace

from src.core.orientation import ALL_ORIENTATIONS, DEFAULT_ORIENTATION, Orientation


@dataclass(frozen=True)
class Placement:
    in_truck: bool
    x: int = 0  # left-bottom-back corner, ignored when outside the truck
    y: int = 0
    z: int = 0
    orientation: Orientation = DEFAULT_ORIENTATION
    truck_id: int = 0

    def __post_init__(self):
        if type(self.in_truck) is not bool:
            raise ValueError("in_truck must be a boolean")
        if any(type(v) is not int for v in (self.x, self.y, self.z)):
            raise ValueError("placement coordinates must be integers")
        if not isinstance(self.orientation, tuple) or self.orientation not in ALL_ORIENTATIONS:
            raise ValueError("orientation must be a permutation of x, y, z")


State = tuple[Placement, ...]


def take_out(placement: Placement) -> Placement:
    return replace(placement, in_truck=False, x=0, y=0, z=0, truck_id=0)
```

3.3 Batasan dan Fungsi Objektif

`find_violations` memeriksa kelima batasan untuk semua paket di dalam truk. Balok direpresentasikan sebagai interval setengah terbuka $[x_0, x_1)$, sehingga dua paket yang hanya bersentuhan sisi tidak dianggap bertumpuk. Fungsi `rests_on` mengikuti definisi tumpuan pada spesifikasi: paket atas tepat berada di permukaan atas paket bawah dan proyeksi alasnya beririsan.

Kode 3.3: Pemeriksaan batasan (`src/core/constraints.py`)

```python
def find_violations(problem: Problem, state: State) -> list[Violation]:
    if len(state) != len(problem.packages):
        return [Violation("state", ())]
    found = []
    groups = [[] for _ in range(problem.truck_count)]
    for i, p in enumerate(state):
        if p.in_truck:
            if (not isinstance(p.truck_id, int) or isinstance(p.truck_id, bool)
                    or not 0 <= p.truck_id < problem.truck_count):
                found.append(Violation("truck", (i,)))
            else:
                groups[p.truck_id].append(i)
    for inside in groups:
        found.extend(_truck_violations(problem, state, inside))
    return found


def _truck_violations(problem: Problem, state: State, inside: list[int]) -> list[Violation]:
    truck = problem.truck
    boxes = {i: box(problem.packages[i], state[i]) for i in inside}
    found: list[Violation] = []

    for i in inside:
        if not within_walls(boxes[i], truck):
            found.append(Violation("wall", (i,)))
```

```
    for i, j in combinations(inside, 2):
        if overlaps(boxes[i], boxes[j]):
            found.append(Violation("overlap", (i, j)))

    # Floating: no point of the base is on the floor or on another package.
    for i in inside:
        if boxes[i][2] > 0 and not any(
            rests_on(boxes[j], boxes[i]) for j in inside if j != i
        ):
            found.append(Violation("floating", (i,)))

    # Fragile packages must not support anything, even a package that also
    # rests on something else ("barang pecah-belah tidak boleh menjadi support").
    for j in inside:
        if problem.packages[j].fragile:
            for i in inside:
                if i != j and rests_on(boxes[j], boxes[i]):
                    found.append(Violation("fragile", (j, i)))

    total_weight = sum(problem.packages[i].weight for i in inside)
    if total_weight > truck.max_capacity:
        found.append(
            Violation("capacity", tuple(inside), total_weight - truck.max_capacity)
        )

    return found
```

Fungsi objektif adalah total *value* paket di dalam truk. Modul ini juga menyediakan `evaluate`, yaitu *value* dikurangi penalti sebesar (*value* paket termahal $+1$) untuk setiap pelanggaran. Dengan penalti tersebut, memasukkan paket secara ilegal tidak pernah lebih menguntungkan daripada mengeluarkannya. Namun ketiga algoritma pada akhirnya selalu bekerja pada *state* yang sudah di-*repair* (Subbab 3.4), sehingga nilai objektif yang dilaporkan sama dengan total *value state* valid.

Kode 3.4: Fungsi objektif (`src/core/objective.py`)

```
def default_penalty(problem: Problem) -> int:
    """One more than the most valuable package, so squeezing a package in
    illegally is never better than leaving it out."""
    return max((p.value for p in problem.packages), default=0) + 1


# Objective "eta" (bonus): value / ETA, so a package due sooner counts for more.
def total_value(problem: Problem, state: State) -> float:
    if len(state) != len(problem.packages):
        raise ValueError("state must contain one placement per package")
    if problem.objective == "eta":
        return sum(pkg.value / pkg.eta for pkg, p in zip(problem.packages, state) if p.in_truck)
    return sum(pkg.value for pkg, p in zip(problem.packages, state) if p.in_truck)


def evaluate(problem: Problem, state: State, penalty: float | None = None) -> Evaluation:
    lam = default_penalty(problem) if penalty is None else penalty
    value = total_value(problem, state)
    violations = tuple(find_violations(problem, state))
    score = value - lam * sum(v.amount for v in violations)
    return Evaluation(value, violations, score)
```

3.4 Heuristik Repair

State awal acak dan sebagian besar tetangga acak melanggar batasan (paket melayang atau bertumpuk). Agar pencarian selalu berada di ruang solusi valid, setiap

state dilewatkan ke `repair`. Heuristik ini deterministik dan hanya *mengeluarkan* atau *menurunkan* paket, tidak pernah menambah paket, sehingga peningkatan nilai tetap berasal dari algoritma pencarian. Langkahnya:

1. Paket yang menembus dinding dikeluarkan dari truk.
2. Selama total berat melebihi kapasitas, paket dengan rasio *value*/berat terendah dikeluarkan.
3. Paket diproses dari bawah ke atas (urut z, lalu *value* terbesar). Paket yang bertumpuk dengan paket yang sudah diterima dikeluarkan. Paket lain "dijatuhkan" ke permukaan tertinggi di bawahnya (atau lantai), seperti gravitasi. Jika permukaan tempatnya mendarat adalah paket *fragile*, paket dikeluarkan.

Kode 3.5: Heuristik *repair* untuk satu truk (`src/core/repair.py`)

```python
def _repair_truck(problem: Problem, state: State, truck_id: int) -> State:
    packages = problem.packages
    truck = problem.truck
    placements = list(state)

    def inside() -> list[int]:
        return [i for i, p in enumerate(placements) if p.in_truck and p.truck_id == truck_id]

    # 1. Walls
    for i in inside():
        if not within_walls(box(packages[i], placements[i]), truck):
            placements[i] = take_out(placements[i])

    # 2. Capacity
    total_weight = sum(packages[i].weight for i in inside())
    by_ratio = sorted(inside(), key=lambda i: (packages[i].value / packages[i].weight, i))
    for i in by_ratio:
        if total_weight <= truck.max_capacity:
            break
        placements[i] = take_out(placements[i])
        total_weight -= packages[i].weight

    # 3. Geometry + gravity
    order = sorted(inside(), key=lambda i: (placements[i].z, -packages[i].value, i))
    accepted: list[tuple[int, Box]] = []
    for i in order:
        b = box(packages[i], placements[i])
        if any(overlaps(b, other) for _, other in accepted):
            placements[i] = take_out(placements[i])
            continue
        below = [(j, other) for j, other in accepted
                 if footprint_overlaps(b, other) and other[5] <= b[2]]
        landing_z = max((other[5] for _, other in below), default=0)
        supporters = [j for j, other in below if other[5] == landing_z and landing_z > 0]
        if any(packages[j].fragile for j in supporters):
            placements[i] = take_out(placements[i])
            continue
        placements[i] = replace(placements[i], z=landing_z)
        accepted.append((i, box(packages[i], placements[i])))

    return tuple(placements)
```

3.5 Aksi Tetangga

Ketiga aksi pada spesifikasi diimplementasikan sebagai fungsi murni yang mengembalikan *state* baru: `move` memindahkan paket ke koordinat lain atau ke luar truk,

`rotate` memutar paket 90° terhadap suatu sumbu (dua sumbu lainnya bertukar), dan `swap` menukar posisi dua paket (masing-masing tetap dengan orientasinya).

Kode 3.6: Aksi *move*, *rotate*, dan *swap* (`src/core/neighbors.py`)

```python
def move(state: State, i: int, target: tuple[int, int, int] | None,
         truck_id: int | None = None) -> State:
    if target is OUTSIDE:
        return _with(state, i, take_out(state[i]))
    x, y, z = target
    destination = state[i].truck_id if truck_id is None else truck_id
    return _with(state, i, replace(state[i], in_truck=True, x=x, y=y, z=z,
                                   truck_id=destination))


def rotate(state: State, i: int, axis: str) -> State:
    """Turn package i 90 degrees about ‘axis‘; its corner stays in place."""
    p = state[i]
    return _with(state, i, replace(p, orientation=rotate_orientation(p.orientation, axis)))


def swap(state: State, i: int, j: int) -> State:
    """Packages i and j trade positions; each keeps its own orientation."""
    a, b = state[i], state[j]
    new_a = replace(a, in_truck=b.in_truck, x=b.x, y=b.y, z=b.z, truck_id=b.truck_id)
    new_b = replace(b, in_truck=a.in_truck, x=a.x, y=a.y, z=a.z, truck_id=a.truck_id)
    if not new_a.in_truck:
        new_a = take_out(new_a)
    if not new_b.in_truck:
        new_b = take_out(new_b)
    return _with(_with(state, i, new_a), j, new_b)
```

Ruang koordinat bilangan bulat sangat besar, sedangkan kebanyakan koordinat acak menghasilkan paket melayang. Karena itu, tujuan aksi *move* diambil dari *extreme points* [9]: titik asal dan titik di kanan, depan, serta atas setiap paket yang sudah ada di truk, selama paket masih muat di dalam dinding. Tetangga acak (`random_neighbor`) memilih jenis aksi secara seragam. Untuk *move*, paket di dalam truk dikeluarkan dengan peluang 0,2; selain itu tujuannya diambil dari *extreme point* (peluang 0,5) atau dari koordinat acak di dalam dinding. `all_neighbors` membangkitkan semua tetangga untuk *steepest-ascent*: semua rotasi, semua *swap* yang mengubah sesuatu, serta semua *move* ke luar truk dan ke setiap *extreme point*.

Kode 3.7: *Extreme points* dan tetangga acak (`src/core/neighbors.py`)

```python
def candidate_positions(problem: Problem, state: State, i: int,
                        truck_id: int | None = None) -> list[tuple[int, int, int]]:
    """Extreme points where package i could go: the origin plus the points
    right of, in front of and on top of every package in the truck, keeping
    only those where package i stays inside the walls.

    Adapted from the extreme-point idea in T. G. Crainic, G. Perboli and
    R. Tadei, "Extreme Point-Based Heuristics for Three-Dimensional Bin
    Packing", INFORMS Journal on Computing 20(3), 2008.
    """
    truck = problem.truck
    destination = state[i].truck_id if truck_id is None else truck_id
    dx, dy, dz = extents(problem.packages[i], state[i].orientation)
    points = {(0, 0, 0)}
    for j, p in enumerate(state):
        if p.in_truck and p.truck_id == destination and j != i:
            x0, y0, z0, x1, y1, z1 = box(problem.packages[j], p)
            points.update({(x1, y0, z0), (x0, y1, z0), (x0, y0, z1)})
    return sorted(
        (x, y, z) for x, y, z in points
        if x + dx <= truck.w and y + dy <= truck.l and z + dz <= truck.h
    )
```

```python

def random_position(problem: Problem, state: State, i: int,
                    rng: random.Random, truck_id: int | None = None) -> tuple[int, int, int] | None:
    """Half the time an extreme point, otherwise a uniform in-bounds corner.
    None when package i does not fit in the truck in its orientation."""
    if rng.random() < 0.5:
        candidates = candidate_positions(problem, state, i, truck_id)
        if candidates:
            return rng.choice(candidates)
    truck = problem.truck
    dx, dy, dz = extents(problem.packages[i], state[i].orientation)
    if dx > truck.w or dy > truck.l or dz > truck.h:
        return None
    return (rng.randint(0, truck.w - dx), rng.randint(0, truck.l - dy),
            rng.randint(0, truck.h - dz))


def random_neighbor(problem: Problem, state: State,
                    rng: random.Random) -> tuple[State, Move]:
    """One random successor (stochastic HC, SA, GA mutation).

    The move type is uniform among move/rotate/swap. For "move", an inside
    package goes out with probability 0.2, otherwise to a random_position.
    """
    n = len(state)
    if not n:
        raise ValueError("cannot generate a neighbor for an empty state")
    kind = rng.choice(("move", "rotate", "swap") if n >= 2 else ("move", "rotate"))
    if kind == "rotate":
        i, axis = rng.randrange(n), rng.choice(AXES)
        return rotate(state, i, axis), Move("rotate", i, axis=axis)
    if kind == "swap":
        i, j = rng.sample(range(n), 2)
        return swap(state, i, j), Move("swap", i, j=j)
    i = rng.randrange(n)
    truck_id = state[i].truck_id
    if state[i].in_truck and rng.random() < 0.2:
        target = OUTSIDE
    else:
        if problem.truck_count > 1:
            truck_id = rng.randrange(problem.truck_count)
        target = random_position(problem, state, i, rng, truck_id)
    return move(state, i, target, truck_id), Move("move", i, target=target, truck_id=truck_id)
```

3.6 State Awal

Sesuai spesifikasi, *state* awal dibangkitkan acak: setiap paket mendapat orientasi acak dan, dengan peluang 0,5, dimasukkan ke truk pada koordinat acak di dalam dinding. *State* ini umumnya tidak valid, lalu di-*repair* oleh setiap algoritma.

Kode 3.8: *State* awal acak (`src/core/initial.py`)

```python
def random_state(problem: Problem, rng: random.Random, p_in: float = 0.5) -> State:
    truck = problem.truck
    placements = []
    for package in problem.packages:
        orientation = rng.choice(ALL_ORIENTATIONS)
        dx, dy, dz = extents(package, orientation)
        fits = dx <= truck.w and dy <= truck.l and dz <= truck.h
        if fits and rng.random() < p_in:
            truck_id = rng.randrange(problem.truck_count) if problem.truck_count > 1 else 0
            placements.append(Placement(
                True,
                rng.randint(0, truck.w - dx),
```

```
                rng.randint(0, truck.l - dy),
                rng.randint(0, truck.h - dz),
                orientation,
                truck_id,
            ))
        else:
            placements.append(Placement(False, orientation=orientation))
    return tuple(placements)
```

3.7 Batas Atas Optimum Global

Optimum global sebenarnya tidak diketahui. Untuk menilai seberapa dekat hasil setiap algoritma, dihitung batas atas berupa *knapsack* 0/1 dengan dua batasan, yaitu berat (kapasitas truk) dan volume (volume peti kemas), yang diselesaikan secara tepat dengan *dynamic programming* [10]. Relaksasi ini mengabaikan geometri, sehingga nilainya selalu $\geq$ optimum global. Besaran gap $= 1 - f(s)/\text{batas atas}$ merupakan batas atas jarak hasil ke optimum global.

Kode 3.9: Batas atas *knapsack* (`src/core/bounds.py`)

```
def value_upper_bound(problem: Problem) -> float:
    t = problem.truck
    packages = [p for i, p in enumerate(problem.packages)
                if _fits_somehow(problem, i) and p.weight <= t.max_capacity]
    volume = min(t.w * t.l * t.h * problem.truck_count,
                 sum(p.w * p.l * p.h for p in packages))
    capacity = min(t.max_capacity * problem.truck_count, sum(p.weight for p in packages))
    def value(p):
        return p.value / p.eta if problem.objective == "eta" else p.value

    if (capacity == sum(p.weight for p in packages)
            and volume == sum(p.w * p.l * p.h for p in packages)):
        return sum(value(p) for p in packages)
    best = np.zeros((capacity + 1, volume + 1), dtype=np.float64)
    for p in packages:
        v = p.w * p.l * p.h
        with_p = best[:-p.weight or None, :-v or None] + value(p)
        np.maximum(best[p.weight:, v:], with_p, out=best[p.weight:, v:])
    return float(best[-1, -1])
```

3.8 Hill-Climbing

Semua varian Hill-Climbing diimplementasikan di `src/algorithms/hill_climbing.py` (bonus). Setiap tetangga di-*repair* sebelum dinilai. Parameter yang dipakai pada eksperimen: `max_iterations`=100, `neighbor_samples`=50 untuk *stochastic*, `max_sideways`=20, dan `max_restarts`=5 (instans *large* memakai 2 karena waktu eksekusinya lama).

- **Steepest-ascent** memakai `_best_neighbor`, yang menilai seluruh `all_neighbors`. Berhenti jika tetangga terbaik tidak lebih baik.
- **Stochastic** memilih secara acak salah satu tetangga yang lebih baik. Untuk menghemat waktu, mula-mula diambil 50 tetangga acak. Jika tidak ada yang lebih baik, seluruh `all_neighbors` diperiksa dan salah satu tetangga yang

lebih baik dipilih secara seragam. Pencarian berhenti hanya jika tidak ada satu pun tetangga yang lebih baik (*local optimum*) atau `max_iterations` tercapai.

- **Sideways move** seperti *steepest-ascent*, tetapi boleh pindah ke tetangga bernilai sama yang belum pernah dikunjungi. Berhenti ketika banyak *sideways move* mencapai `max_sideways`. Karena tetangga bernilai sama yang sudah dikunjungi tidak dipilih lagi, pencarian tidak dapat berputar dalam siklus.
- **Random-restart** menjalankan *steepest-ascent* dari *state* awal acak sebanyak `max_restarts`+1 kali dan mengambil hasil terbaik. Banyak iterasi per *restart* dicatat.

Kode 3.10: Satu kali pendakian Hill-Climbing (`src/algorithms/hill_climbing.py`)

```
def _run_single_climb(
    problem: Problem,
    params: HCParams,
    rng: random.Random,
    start_state: State,
    variant: str,
) -> _ClimbOutcome:
    current, current_value = _repair_value(problem, start_state)
    initial_state = current
    best_value = current_value
    values = [current_value]
    best_values = [best_value]
    seen = {current}
    sideways_moves = 0
    stop_reason = "max_iterations"

    for _ in range(1, params.max_iterations + 1):
        if variant == "stochastic":
            candidate = _random_improving_neighbor(
                problem, current, current_value, params.neighbor_samples, rng
            )
            if candidate is None:
                stop_reason = "local_optimum"
                break
        else:
            if variant == "sideways":
                candidate = _best_neighbor(problem, current, current_value, seen)
            else:
                candidate = _best_neighbor(problem, current)
            if candidate is None:
                stop_reason = "local_optimum"
                break
            if candidate.value < current_value:
                stop_reason = "local_optimum"
                break
            if candidate.value == current_value:
                if variant != "sideways":
                    stop_reason = "local_optimum"
                    break
                if sideways_moves >= params.max_sideways:
                    stop_reason = "max_sideways"
                    break
                sideways_moves += 1

        current, current_value = candidate.state, candidate.value
        seen.add(current)
        best_value = max(best_value, current_value)
        values.append(current_value)
        best_values.append(best_value)
        if variant == "sideways" and sideways_moves > 0 and sideways_moves >= params.max_sideways:
```

```
            stop_reason = "max_sideways"
            break
    return _ClimbOutcome(
        initial_state=initial_state,
        initial_value=values[0],
        final_state=current,
        final_value=current_value,
        values=values,
        best_values=best_values,
        iterations=len(values) - 1,
        stop_reason=stop_reason,
        sideways_moves=sideways_moves,
    )
```

3.9 Simulated Annealing

Simulated Annealing diimplementasikan di `src/algorithms/simulated_annealing.py`. Setiap iterasi mengambil satu tetangga acak yang sudah di-*repair*. Tetangga diterima jika $\Delta E \geq 0$; jika tidak, diterima dengan peluang $e^{\Delta E/T}$. Heuristik yang dipakai:

- **Kalibrasi suhu awal.** Jika T_0 tidak diberikan, diambil 100 tetangga acak dari *state* awal dan dihitung rata-rata $|\Delta E|$ langkah yang memburuk, lalu $T_0 = -\overline{|\Delta E|}/\ln p_0$. Dengan begitu, langkah memburuk rata-rata diterima dengan peluang p_0 di awal pencarian [11].
- **Jadwal dengan titik ujung tetap.** Iterasi pertama memakai T_0 dan iterasi terakhir tepat $T_{\min}$, secara geometrik ($T_k = T_0(T_{\min}/T_0)^{k/(n-1)}$, $k = 0, \ldots, n-1$) atau linear. Banyak iterasi menjadi satu-satunya parameter biaya.
- **Simpan *state* terbaik.** Hasil akhir adalah *state* terbaik yang pernah dikunjungi, bukan *state* terakhir.
- **Ratio swap (opsional).** Dengan peluang `ratio_bias`, tetangga yang diusulkan adalah *swap* antara paket di dalam truk yang rasio *value*/beratnya rendah dan paket di luar yang rasionya tinggi. Aksinya tetap *swap* biasa; hanya pemilihan paketnya yang dibias.
- **Definisi *stuck*.** SA dianggap *stuck* di *local optimum* jika nilai objektif *state* sekarang tidak berubah selama 500 iterasi berturut-turut. Satu *plateau* dihitung satu kali.

Kode 3.11: Kalibrasi T_0, jadwal suhu, dan peluang penerimaan (`src/algorithms/simulated_annealing.py`)

```
# e^(dE/T) for a worse move, 1 for an equal or better one.
def acceptance_probability(delta: float, t: float) -> float:
    if delta >= 0:
        return 1.0
    return math.exp(delta / t)


# Temperature at iteration k of n: t0 at k = 0 and t_min at k = n.
def temperature(k: int, n: int, t0: float, t_min: float, schedule: str) -> float:
    frac = k / n
```

```
    if k == 0:
        return t0
    if k == n:
        return t_min
    if schedule == "geometric":
        value = math.exp(math.log(t0) + frac * (math.log(t_min) - math.log(t0)))
    else:
        value = t_min + (t0 - t_min) * (1 - frac)
    return min(t0, max(t_min, value))
# t0 such that an average worse move is accepted with probability p0 (Ben-Ameur 2004).
def calibrate_t0(problem: Problem, state: State, p0: float, samples: int,
                 rng: random.Random) -> tuple[float, float]:
    value = total_value(problem, state)
    worse = []
    for _ in range(samples):
        _, candidate_value = _neighbor(problem, state, rng)
        if candidate_value < value:
            worse.append(value - candidate_value)
    if not worse:
        return 1.0, 0.0
    mean_worse = sum(worse) / len(worse)
    return -mean_worse / math.log(p0), mean_worse
```

Kode 3.12: Iterasi utama Simulated Annealing (`src/algorithms/simulated_annealing.py`)

```
def run_simulated_annealing(problem: Problem, params: SAParams, seed: int) -> RunResult:
    rng = random.Random(seed)
    start = time.perf_counter()

    current = repair(problem, random_state(problem, rng))
    current_value = total_value(problem, current)
    initial_state, initial_value = current, current_value

    if params.t0 is None:
        t0, mean_worse = calibrate_t0(problem, current, params.p0,
                                      params.calibration_samples, rng)
        if t0 <= params.t_min:
            t0 = math.nextafter(params.t_min, math.inf)
        require_number(t0, "calibrated t0")
    else:
        t0, mean_worse = params.t0, None
    t_min = params.t_min
    n = params.max_iterations

    best_state, best_value, best_iteration = current, current_value, 0
    history = {"value": [current_value], "best": [current_value],
               "temperature": [], "eET": [], "log_eET": []}
    accepted = accepted_worse = null_moves = 0
    plateau = stuck_count = stuck_iterations = 0
    schedule_steps = max(n - 1, 1)

    for k in range(n):
        t = temperature(k, schedule_steps, t0, t_min, params.schedule)
        candidate, candidate_value = _neighbor(problem, current, rng, params.ratio_bias)
        delta = candidate_value - current_value
        # Metropolis rule: accept dE >= 0, otherwise with probability e^(dE/T).
        p = acceptance_probability(delta, t)
        moved_value = False
        if delta >= 0 or rng.random() < p:
            accepted += 1
            accepted_worse += delta < 0
            null_moves += candidate == current
            moved_value = delta != 0
            current, current_value = candidate, candidate_value

        # Stuck: value unchanged for stuck_patience iterations, counted once per plateau.
        if moved_value:
            plateau = 0
```

```
        else:
            plateau += 1
            if plateau == params.stuck_patience:
                stuck_count += 1
                stuck_iterations += plateau
            elif plateau > params.stuck_patience:
                stuck_iterations += 1

        if current_value > best_value:
            best_state, best_value, best_iteration = current, current_value, k + 1
        history["value"].append(current_value)
        history["best"].append(best_value)
        history["temperature"].append(t)
        history["eET"].append(p)
        history["log_eET"].append(delta / t)

    return RunResult(
        algorithm="sa",
        params=asdict(params),
        seed=seed,
        duration_s=time.perf_counter() - start,
        initial_state=initial_state,
        initial_value=initial_value,
        final_state=best_state,
        final_value=best_value,
        history=history,
        extra={
            "t0": t0, "t_min": t_min, "mean_worse_delta": mean_worse,
            "alpha": math.exp((math.log(t_min) - math.log(t0)) / schedule_steps)
            if params.schedule == "geometric" else None,
            "accepted": accepted, "rejected": n - accepted,
            "accepted_worse": accepted_worse, "null_moves": null_moves,
            "stuck_count": stuck_count, "stuck_iterations": stuck_iterations,
            "stuck_definition": "unchanged_objective_for_stuck_patience_proposals",
            "best_iteration": best_iteration, "final_current_value": current_value,
        },
    )
```

3.10 Genetic Algorithm

Genetic Algorithm diimplementasikan di `src/algorithms/genetic.py` mengikuti salindia kuliah [8]:

- **Individu** adalah satu *state* lengkap; gen ke-i adalah `Placement` paket ke-i.
- **Populasi awal** berisi k *state* acak yang di-*repair*.
- **Fitness** adalah total *value* (fungsi objektif).
- **Seleksi** memakai *roulette wheel* (`roulette_select`).
- **Crossover** memakai *single-point crossover*: titik potong acak di $[1, n-1]$, kedua induk bertukar ekor sehingga terbentuk dua anak.
- **Mutasi**: dengan peluang 0,2, anak dikenai satu aksi tetangga acak (*move*/*rotate*/*swap*).

Ada tiga adaptasi terhadap versi salindia. Pertama, setiap anak di-*repair*, karena hasil *crossover* dapat bertumpuk atau melayang. Kedua, satu individu terbaik

dibawa langsung ke generasi berikutnya (*elitism*). Ketiga, mutasi memakai aksi tetangga, bukan mengganti satu karakter *string*, karena gen berisi posisi dan orientasi paket.

Kode 3.13: Seleksi dan *crossover* (`src/algorithms/genetic.py`)

```python
def roulette_select(population: Sequence[T], fitness: Sequence[float],
                    rng: random.Random) -> T:
    """Fitness-proportionate selection; uniform if every fitness is 0."""
    if sum(fitness) <= 0:
        return rng.choice(population)
    return rng.choices(population, weights=fitness, k=1)[0]


def single_point_crossover(p1: tuple, p2: tuple,
                           rng: random.Random) -> tuple[tuple, tuple]:
    """Cut both parents at the same random point in [1, n-1] and swap tails."""
    n = len(p1)
    if n < 2:
        return p1, p2
    cut = rng.randint(1, n - 1)
    return p1[:cut] + p2[cut:], p2[:cut] + p1[cut:]
```

Kode 3.14: Iterasi utama Genetic Algorithm (`src/algorithms/genetic.py`)

```python
def run_genetic(problem: Problem, params: GAParams, seed: int) -> RunResult:
    rng = random.Random(seed)
    start = time.perf_counter()

    def fitness_of(population):
        return [total_value(problem, individual) for individual in population]

    population = [repair(problem, random_state(problem, rng))
                  for _ in range(params.population_size)]
    fitness = fitness_of(population)

    # "Initial state" for the report: the best individual of generation 0.
    best_index = max(range(len(population)), key=fitness.__getitem__)
    initial_state, initial_value = population[best_index], fitness[best_index]
    best_state, best_value, best_generation = initial_state, initial_value, 0
    history = {"max": [max(fitness)], "mean": [fmean(fitness)]}

    for generation in range(1, params.generations + 1):
        ranked = sorted(range(len(population)), key=fitness.__getitem__, reverse=True)
        next_population = [population[i] for i in ranked[:params.elitism]]
        while len(next_population) < params.population_size:
            p1 = roulette_select(population, fitness, rng)
            p2 = roulette_select(population, fitness, rng)
            for child in single_point_crossover(p1, p2, rng):
                if child and rng.random() < params.mutation_rate:
                    child, _ = random_neighbor(problem, child, rng)
                next_population.append(repair(problem, child))
        population = next_population[:params.population_size]
        fitness = fitness_of(population)

        history["max"].append(max(fitness))
        history["mean"].append(fmean(fitness))
        generation_best = max(range(len(population)), key=fitness.__getitem__)
        if fitness[generation_best] > best_value:
            best_state = population[generation_best]
            best_value = fitness[generation_best]
            best_generation = generation

    return RunResult(
        algorithm="ga",
        params=asdict(params),
        seed=seed,
        duration_s=time.perf_counter() - start,
        initial_state=initial_state,
```

```
        initial_value=initial_value,
        final_state=best_state,
        final_value=best_value,
        history=history,
        extra={"best_generation": best_generation},
    )
```

3.11 Bonus: Fungsi Objektif Berbasis ETA

Sebagai fungsi objektif alternatif tanpa menambah variabel, *value* setiap paket dibagi dengan ETA-nya:

$$f_{\text{ETA}}(s) = \sum_{i\,:\,\text{paket } i \text{ di dalam truk}} \frac{\text{value}_i}{\text{ETA}_i}. \tag{3.1}$$

Paket yang harus tiba lebih cepat (ETA kecil) menjadi lebih berharga, sehingga truk memprioritaskan paket yang mendesak. Objektif ini dipilih dengan opsi `-objective eta` dan berlaku untuk ketiga algoritma.

3.12 Bonus: Multi-Truk

Program dapat menerima armada berisi N truk identik (opsi `-trucks N` atau `"truck_count"` pada JSON instans). Setiap truk memakai dimensi dan kapasitas `truck` dari instans. Objektifnya adalah total *value* paket yang diangkut seluruh armada, dan setiap paket hanya dapat berada di satu truk. Perubahan utamanya:

- **Representasi.** `Placement` mendapat atribut `truck_id`; `Problem` mendapat `truck_count`.
- **Batasan dan *repair*** dijalankan terpisah untuk setiap truk. Dinding, tumpang tindih, tumpuan, *fragile*, dan kapasitas berat hanya membandingkan paket di truk yang sama. Paket dengan `truck_id` tidak valid dikeluarkan.
- **Aksi tetangga.** *Move* dapat memindahkan paket ke truk lain (tujuannya *extreme point* di truk tujuan), dan *swap* menukar posisi sekaligus truk dua paket. `all_neighbors` membangkitkan *move* ke setiap truk. Ketiga algoritma tidak perlu diubah karena bekerja melalui operator yang sama.
- **Batas atas** memakai kapasitas berat dan volume gabungan armada, tetap sebagai relaksasi *knapsack*.
- **Visualisasi** menampilkan setiap truk beserta berat muatannya.

Untuk satu truk, urutan bilangan acak yang dipakai tidak berubah, sehingga seluruh hasil eksperimen satu truk tetap dapat direproduksi persis.

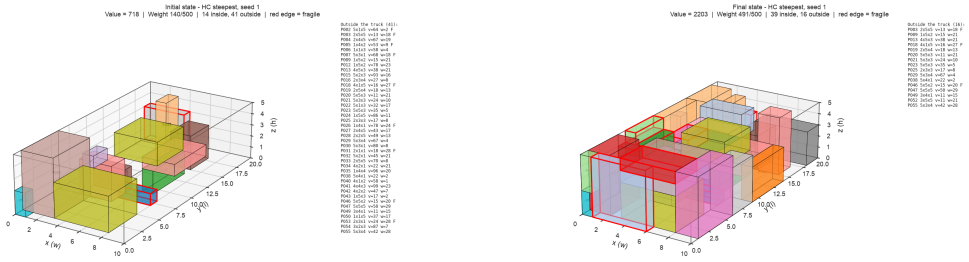
Kode 3.15: *Repair* per truk (`src/core/repair.py`)

```
def repair(problem: Problem, state: State) -> State:
    if len(state) != len(problem.packages):
```

```
        raise ValueError("state must contain one placement per package")
    state = tuple(
        take_out(p) if p.in_truck and (
            not isinstance(p.truck_id, int) or isinstance(p.truck_id, bool)
            or not 0 <= p.truck_id < problem.truck_count
        ) else p
        for p in state
    )
    for truck_id in range(problem.truck_count):
        state = _repair_truck(problem, state, truck_id)
    return state
```

3.13 Visualisasi dan Keluaran

Setiap *run* menyimpan satu folder di `output/` berisi `result.json` (parameter, *seed*, durasi, *state* awal dan akhir, riwayat nilai), `initial.png` dan `final.png` (visualisasi 3D *state*), serta `curve.png` (plot objektif terhadap iterasi). Untuk SA ada tambahan `eet.png` (plot $e^{\Delta E/T}$). Visualisasi 3D menampilkan isi peti kemas (paket *fragile* bertepi merah), nilai *state*, total berat, dan daftar paket di luar truk (Gambar 3.1). Skrip eksperimen juga menyimpan `summary.csv` dan plot pembanding.



(a) *State* awal (b) *State* akhir

Gambar 3.1: Visualisasi *state* awal dan akhir (HC *steepest-ascent*, instans *medium*, *seed* 1)

Program dijalankan melalui `src/main.py` untuk satu kali *run*, misalnya:

```
python -m src.main hc --instance data/instances/medium.json --variant steepest --seed 1
python -m src.main sa --instance data/instances/medium.json --iters 20000 --p0 0.01
python -m src.main ga --instance data/instances/medium.json --pop 50 --gens 100
```

Eksperimen lengkap dijalankan dengan modul `src.experiments.hc_experiment`, `sa_experiment`, dan `ga_experiment`.

Bab 4

Pembahasan: Hasil dan Analisis Eksperimen

4.1 Metodologi Eksperimen

Setiap konfigurasi dijalankan 3 kali (*seed* 1, 2, 3) pada ketiga instans. Setiap pasangan (konfigurasi, *seed*) mendapat *seed* acak tersendiri yang diturunkan dari nama konfigurasinya (`run_seed`). Dengan demikian, misalnya, GA 50 generasi bukan potongan awal dari GA 200 generasi, dan setiap *run* dapat direproduksi persis. Durasi diukur dengan `time.perf_counter` untuk satu *run* dalam satu proses Python. Seluruh angka pada bab ini diambil dari `output/*/*/summary.csv`. Folder setiap *run* berisi *state* awal dan akhir (JSON dan gambar 3D) serta plot objektif terhadap iterasi.

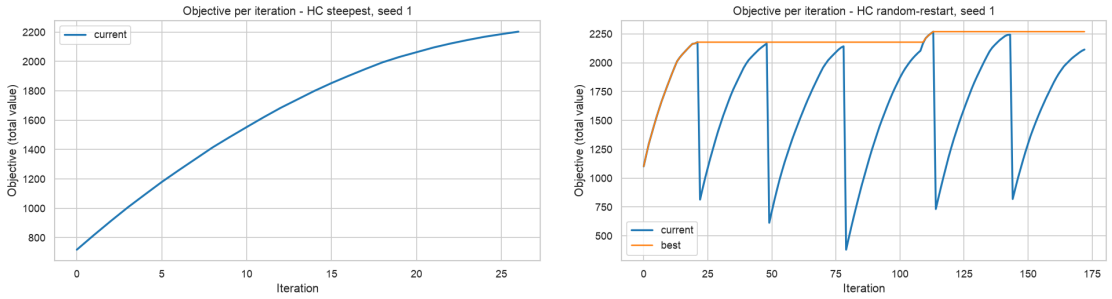
Kolom **gap** adalah $1 - f(s)/\text{batas atas}$ (Subbab 3.7), yaitu batas atas jarak hasil ke optimum global. Gap 0% berarti hasilnya terbukti optimum global.

4.2 Hasil Hill-Climbing

Tabel 4.1 merangkum keempat varian. Hasil setiap *run*, termasuk banyak iterasi, alasan berhenti, serta banyak *restart* dan iterasi per *restart*, ada pada Tabel 4.2.

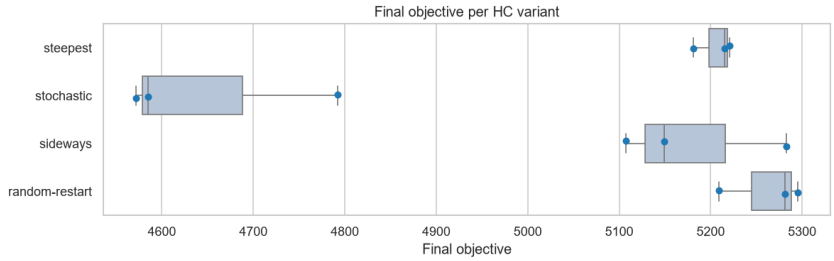
Tabel 4.1: Ringkasan Hill-Climbing (rata-rata 3 *run*)

Instans	Varian	Akhir (mean ± std)	Min–maks	Gap (%)	Durasi (s)	Iterasi
small	Steepest	680.7 ± 24.4	661–708	7.3	0.29	8.3
	Stochastic	677.0 ± 53.7	615–708	7.8	0.10	11.0
	Sideways	682.0 ± 45.0	630–708	7.1	1.18	28.7
	Random-restart	712.7 ± 8.1	708–722	2.9	1.21	40.7
medium	Steepest	2213.7 ± 23.9	2197–2241	7.1	53.51	29.3
	Stochastic	2105.3 ± 56.9	2040–2144	11.7	19.42	48.0
	Sideways	2192.7 ± 12.7	2178–2201	8.0	50.77	47.7
	Random-restart	2260.7 ± 7.2	2256–2269	5.2	120.72	167.7
large	Steepest	5205.7 ± 21.6	5181–5221	7.6	693.51	64.0
	Stochastic	4649.7 ± 123.4	4572–4792	17.5	600.95	98.7
	Sideways	5179.7 ± 91.9	5107–5283	8.0	922.48	82.3
	Random-restart	5261.7 ± 46.1	5209–5295	6.6	1683.70	186.3



(a) *Steepest-ascent* (b) *Random-restart*

Gambar 4.1: Nilai objektif terhadap iterasi Hill-Climbing (instans *medium*, *seed* 1)



Gambar 4.2: Sebaran nilai akhir varian Hill-Climbing (instans *large*)

Tabel 4.2: Hasil setiap *run* Hill-Climbing

Varian	Seed	Awal	Akhir	Gap (%)	Durasi (s)	Iterasi	Berhenti karena
small (batas atas 734)							
Steepest	1	119	708	3.5	0.17	11	local optimum
Steepest	2	222	661	10.0	0.20	7	local optimum
Steepest	3	209	673	8.3	0.49	7	local optimum
Stochastic	1	270	615	16.2	0.10	10	local optimum
Stochastic	2	347	708	3.5	0.09	13	local optimum

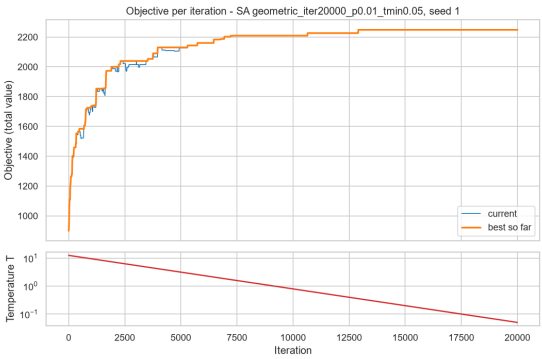
Varian	Seed	Awal	Akhir	Gap (%)	Durasi (s)	Iterasi	Berhenti karena
Stochastic	3	317	708	3.5	0.10	10	local optimum
Sideways	1	235	630	14.2	1.27	27	max sideways
Sideways	2	189	708	3.5	1.20	29	max sideways
Sideways	3	279	708	3.5	1.07	30	max sideways
Random-restart	1	434	708	3.5	0.99	37	max restart (5 restart: 4, 8, 7, 7, 5, 6)
Random-restart	2	151	722	1.6	1.28	46	max restart (5 restart: 8, 6, 9, 4, 10, 9)
Random-restart	3	212	708	3.5	1.36	39	max restart (5 restart: 8, 7, 6, 3, 7, 8)
medium (batas atas 2384)							
Steepest	1	718	2203	7.6	49.65	26	local optimum
Steepest	2	532	2197	7.8	86.02	31	local optimum
Steepest	3	602	2241	6.0	24.84	31	local optimum
Stochastic	1	944	2132	10.6	17.81	36	local optimum
Stochastic	2	361	2144	10.1	23.74	62	local optimum
Stochastic	3	674	2040	14.4	16.71	46	local optimum
Sideways	1	704	2199	7.8	49.59	47	max sideways
Sideways	2	732	2201	7.7	50.30	48	max sideways
Sideways	3	658	2178	8.6	52.40	48	max sideways
Random-restart	1	1102	2269	4.8	126.60	167	max restart (5 restart: 21, 26, 29, 34, 29, 28)
Random-restart	2	702	2256	5.4	117.72	163	max restart (5 restart: 30, 29, 30, 23, 27, 24)
Random-restart	3	794	2257	5.3	117.85	173	max restart (5 restart: 28, 31, 27, 33, 28, 26)
large (batas atas 5633)							
Steepest	1	1485	5221	7.3	878.74	65	local optimum
Steepest	2	1379	5181	8.0	579.39	64	local optimum
Steepest	3	1506	5215	7.4	622.42	63	local optimum
Stochastic	1	1443	4585	18.6	441.50	100	max iterasi
Stochastic	2	1668	4792	14.9	800.63	96	local optimum
Stochastic	3	1475	4572	18.8	560.71	100	max iterasi
Sideways	1	1644	5107	9.3	934.22	82	max sideways
Sideways	2	1584	5283	6.2	838.40	81	max sideways
Sideways	3	1689	5149	8.6	994.83	84	max sideways
Random-restart	1	1989	5281	6.2	1654.99	185	max restart (2 restart: 52, 67, 66)
Random-restart	2	1329	5295	6.0	1698.62	189	max restart (2 restart: 65, 59, 65)
Random-restart	3	1586	5209	7.5	1697.51	185	max restart (2 restart: 61, 61, 63)

4.3 Hasil Simulated Annealing

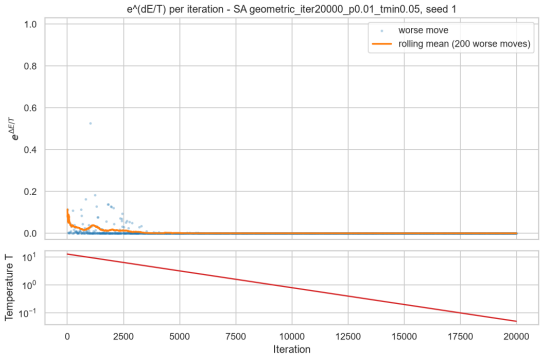
Konfigurasi utama SA memakai jadwal geometrik, 20.000 iterasi, $p_0 = 0{,}01$, dan $T_{\min} = 0{,}05$ (Tabel 4.3). Gambar 4.3 menunjukkan plot objektif dan $e^{\Delta E/T}$ terhadap iterasi. Kolom **Stuck** adalah frekuensi SA *stuck* di *local optimum*, yaitu banyaknya *plateau* dengan nilai objektif tidak berubah selama minimal 500 iterasi.

Tabel 4.3: Hasil setiap *run* Simulated Annealing (geometrik, 20.000 iterasi, $p_0 = 0{,}01$, $T_{\min} = 0{,}05$)

Seed	Awal	Akhir	Gap (%)	Durasi (s)	T_0	Stuck	Iterasi terbaik
small (batas atas 734)							
1	244	734	0.0	4.62	9.28	4	11860
2	94	734	0.0	4.75	9.19	1	776
3	289	734	0.0	4.53	15.12	3	8718
medium (batas atas 2384)							
1	899	2248	5.7	23.12	12.47	4	12907
2	710	2209	7.3	23.24	12.08	7	19857
3	639	2243	5.9	22.75	9.83	8	15616
large (batas atas 5633)							
1	1669	4932	12.4	21.09	11.17	7	18777
2	1505	4943	12.2	21.87	8.11	11	18974
3	1433	5053	10.3	21.56	9.66	7	16325



(a) Nilai objektif dan suhu



(b) $e^{\Delta E/T}$

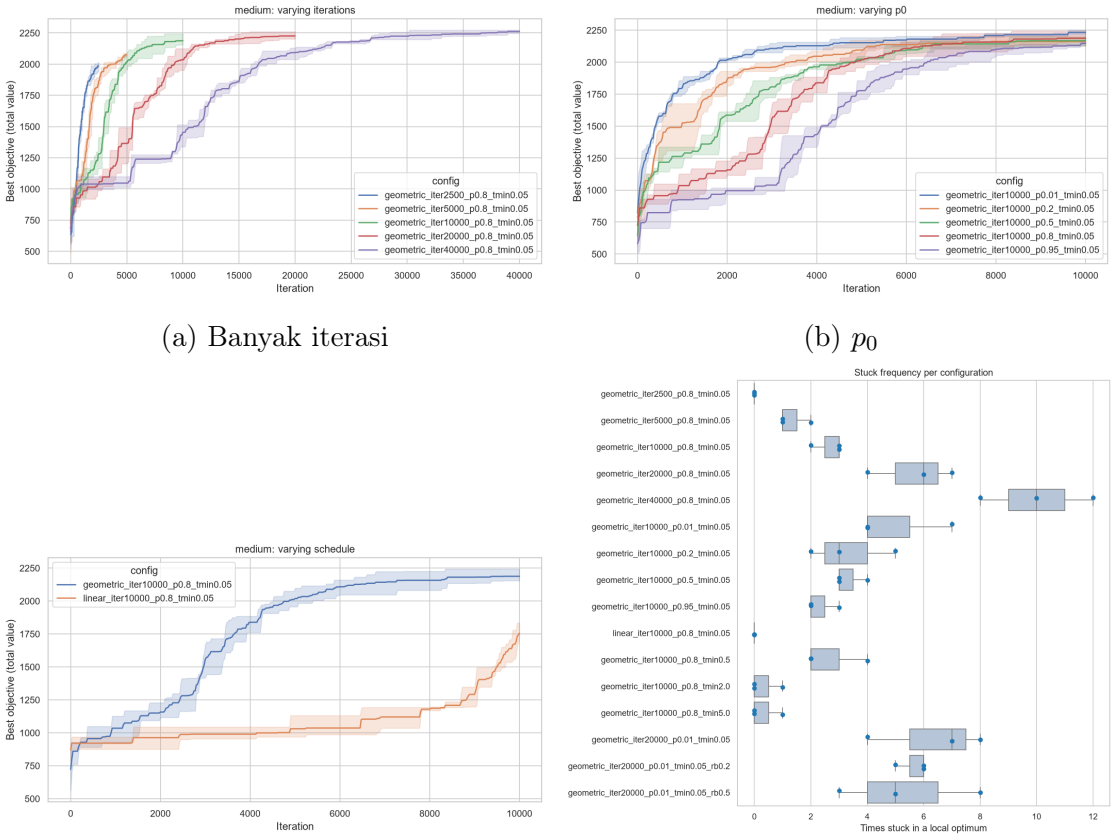
Gambar 4.3: Simulated Annealing pada instans *medium* (*seed* 1)

Parameter SA juga diuji pada instans *medium*, satu parameter divariasikan setiap kali (Tabel 4.4, Gambar 4.4).

Tabel 4.4: Pengaruh parameter Simulated Annealing pada instans *medium* (rata-rata 3 *run*)

Variasi	Jadwal	Iterasi	p_0	$T_{\min}$	Bias	Akhir	Gap (%)	Durasi (s)	Stuck
Iterasi	geometric	2500	0.8	0.05	0	1985.7 ± 41.5	16.7	1.7	0.0
	geometric	5000	0.8	0.05	0	2075.7 ± 14.7	12.9	3.8	1.3
	geometric	10000	0.8	0.05	0	2187.7 ± 49.0	8.2	8.4	2.7
	geometric	20000	0.8	0.05	0	2226.0 ± 30.0	6.6	16.8	5.7
	geometric	40000	0.8	0.05	0	2262.3 ± 13.2	5.1	37.3	10.0
p_0	geometric	10000	0.01	0.05	0	2232.3 ± 18.6	6.4	10.7	5.0
	geometric	10000	0.2	0.05	0	2163.3 ± 37.3	9.3	10.1	3.3
	geometric	10000	0.5	0.05	0	2169.3 ± 39.1	9.0	8.9	3.3
	geometric	10000	0.8	0.05	0	2187.7 ± 49.0	8.2	8.4	2.7
	geometric	10000	0.95	0.05	0	2147.0 ± 13.5	9.9	7.6	2.3
Jadwal	geometric	10000	0.8	0.05	0	2187.7 ± 49.0	8.2	8.4	2.7
	linear	10000	0.8	0.05	0	1755.7 ± 69.0	26.4	3.3	0.0
$T_{\min}$	geometric	10000	0.8	0.05	0	2187.7 ± 49.0	8.2	8.4	2.7
	geometric	10000	0.8	0.5	0	2149.0 ± 79.1	9.9	8.0	2.7
	geometric	10000	0.8	2	0	2132.3 ± 64.6	10.6	6.9	0.3
	geometric	10000	0.8	5	0	1994.3 ± 17.2	16.3	5.6	0.3
Ratio bias	geometric	20000	0.01	0.05	0	2233.3 ± 21.2	6.3	23.0	6.3
	geometric	20000	0.01	0.05	0.2	2247.3 ± 35.0	5.7	22.4	5.7
	geometric	20000	0.01	0.05	0.5	2251.7 ± 40.4	5.5	241.7	5.3

Rata-rata durasi *ratio bias* 0,5 pada Tabel 4.4 dipengaruhi satu *run* (*seed* 1) yang tercatat 683 detik, padahal dua *run* lainnya sekitar 21 detik. Nilai objektifnya tidak terpengaruh.



(a) Banyak iterasi

(b) p_0

(c) Jadwal suhu

(d) Frekuensi *stuck*

Gambar 4.4: Perbandingan konfigurasi Simulated Annealing (instans *medium*)

Tabel 4.5: Pengaruh *ratio bias* pada instans *large* (rata-rata 3 *run*)

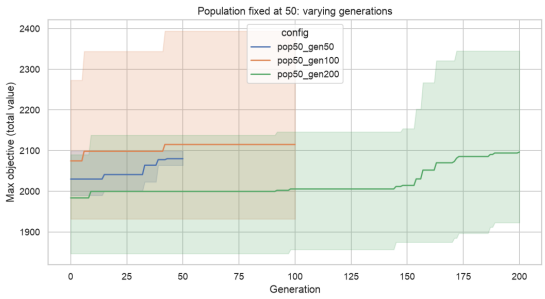
Ratio bias	Akhir	Gap (%)	Durasi (s)	Stuck
0	4976.0 ± 66.9	11.7	21.5	8.3
0.2	5076.3 ± 42.3	9.9	21.3	9.0
0.5	5058.0 ± 74.8	10.2	19.7	8.0

4.4 Hasil Genetic Algorithm

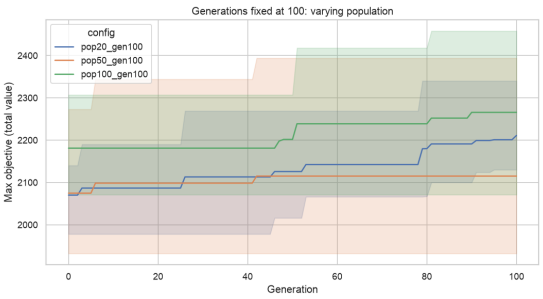
Sesuai spesifikasi, dilakukan dua eksperimen: (1) populasi tetap 50 dengan generasi 50, 100, dan 200; (2) generasi tetap 100 dengan populasi 20, 50, dan 100. Laju mutasi 0,2 dan *elitism* 1. Konfigurasi (50, 100) termasuk dalam kedua eksperimen. Ringkasannya ada pada Tabel 4.6, sedangkan hasil setiap *run* (populasi, iterasi, nilai awal dan akhir, durasi) ada pada Tabel A.1 di Lampiran. Kolom "Awal" adalah nilai individu terbaik generasi ke-0.

Tabel 4.6: Ringkasan Genetic Algorithm (rata-rata 3 *run*)

Instans	Pop.	Gen.	Awal	Akhir (mean ± std)	Min–maks	Gap (%)	Durasi (s)	Gen. terbaik
	50.0	50.0	488.3	608.0 ± 41.2	563–644	17.2	0.21	29.0
	50.0	100.0	486.0	640.7 ± 12.0	629–653	12.7	0.38	61.0
small	50.0	200.0	380.7	698.3 ± 16.7	679–708	4.9	1.03	110.0
	20.0	100.0	389.0	569.3 ± 26.5	548–599	22.4	0.20	64.0
	100.0	100.0	437.0	655.3 ± 11.5	644–667	10.7	1.18	54.7
	50.0	50.0	1082.3	1221.7 ± 58.3	1186–1289	48.8	0.15	24.3
	50.0	100.0	992.0	1319.3 ± 211.7	1154–1558	44.7	0.34	90.3
medium	50.0	200.0	1034.7	1477.7 ± 105.2	1369–1579	38.0	0.67	185.3
	20.0	100.0	975.0	1220.0 ± 98.1	1120–1316	48.8	0.13	89.7
	100.0	100.0	993.3	1422.7 ± 78.7	1353–1508	40.3	0.63	91.3
	50.0	50.0	2030.0	2080.0 ± 17.7	2064–2099	63.1	0.98	27.3
	50.0	100.0	2074.7	2115.0 ± 245.5	1932–2394	62.5	2.42	14.0
large	50.0	200.0	1983.7	2096.3 ± 220.9	1923–2345	62.8	6.37	187.0
	20.0	100.0	2070.0	2211.0 ± 112.9	2130–2340	60.7	1.29	91.3
	100.0	100.0	2181.0	2265.7 ± 193.5	2071–2458	59.8	10.69	57.0

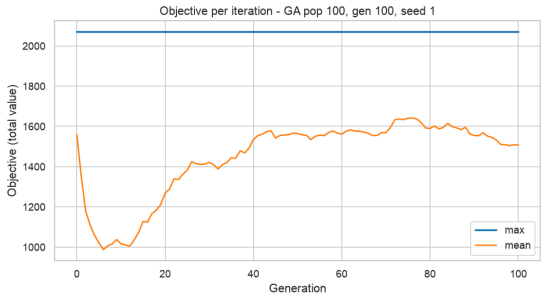


(a) Populasi 50, generasi bervariasi

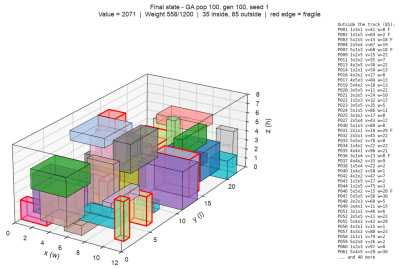


(b) Generasi 100, populasi bervariasi

Gambar 4.5: Nilai objektif maksimum GA terhadap generasi (instans *large*; garis = rata-rata 3 *run*, area = rentang)



(a) Nilai maksimum dan rata-rata populasi



(b) *State* akhir

Gambar 4.6: GA populasi 100, generasi 100 (instans *large*, *seed* 1)

4.5 Hasil Bonus: Objektif ETA

SA (jadwal geometrik, 20.000 iterasi, $p_0 = 0{,}01$, *seed* 1) dijalankan pada instans *medium* dengan objektif ETA dan objektif *value* biasa (`output/sa/medium/eta/`).

Tabel 4.7: Perbandingan objektif *value* dan ETA (SA, instans *medium*, *seed* 1)

Objektif	Total value	$\sum$ value/ETA	Paket dimuat	Rata-rata ETA
value	2266	933,4	40	3,95
ETA	2180	966,0	40	3,70

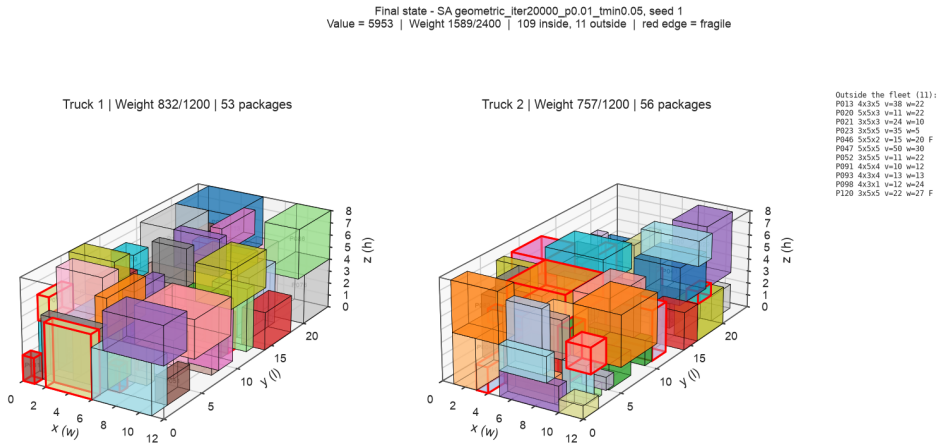
Objektif ETA mengorbankan 3,8% total *value* untuk memuat paket dengan rata-rata ETA lebih kecil (3,70 dibanding 3,95 hari; rata-rata ETA semua paket 4,11). Artinya, paket yang lebih mendesak lebih diprioritaskan, sesuai tujuan objektif ini.

4.6 Hasil Bonus: Multi-Truk

Armada dua truk identik diuji pada instans *medium* dan *large* dengan parameter yang sama seperti eksperimen satu truk (3 *run* per konfigurasi; hasilnya ada di `output/*/medium_trucks2/` dan `output/*/large_trucks2/`). HC hanya dijalankan untuk *steepest* dan *stochastic* di *medium* karena waktu eksekusinya. Untuk kedua instans, total berat dan volume seluruh paket muat di dua truk, sehingga batas atasnya sama dengan total *value* semua paket (2608 dan 6194). Seluruh 42 *state* akhir diperiksa ulang dengan pemeriksa independen berbasis sel, dan semuanya valid.

Tabel 4.8: Hasil multi-truk dibanding satu truk (rata-rata 3 *run*)

Instans	Algoritma	1 truk	2 truk (mean ± std)	Gap 2 truk (%)	Durasi 2 truk (s)
medium	HC *steepest*	2213,7	2544,3 ± 6,5	2,4	48,1
	HC *stochastic*	2105,3	2426,0 ± 51,6	7,0	23,7
	SA (20.000 iterasi, $p_0 = 0{,}01$)	2233,3	2543,0 ± 28,0	2,5	4,6
	GA (pop. 50, gen. 100)	1319,3	1608,7 ± 66,3	38,3	0,7
	GA (pop. 50, gen. 200)	1477,7	1691,7 ± 68,0	35,1	1,3
large	SA (20.000 iterasi, $p_0 = 0{,}01$)	4976,0	5921,7 ± 27,2	4,4	12,2
	GA (pop. 50, gen. 100)	2115,0	2598,7 ± 47,5	58,0	1,0
	GA (pop. 50, gen. 200)	2096,3	2873,0 ± 244,4	53,6	2,2



Gambar 4.7: *State* akhir SA dengan dua truk (instans *large*, *seed* 1)

Dengan dua truk, total *value* yang diangkut naik 14–19% untuk HC dan SA. Kenaikannya tidak 100% karena sebagian besar paket bernilai tinggi sudah terangkut oleh satu truk. Pada *large*, batasnya bukan lagi berat (1589 dari 2400 pada Gambar 4.7), melainkan volume dan geometri: paket yang tersisa di luar umumnya berukuran besar dengan *value* kecil. HC *steepest* dan SA kini hanya 2–4% dari batas atas. Urutan kinerja algoritma tetap sama seperti pada satu truk: SA dan HC *steepest* unggul, HC *stochastic* sedikit di bawahnya (gap 7%), sedangkan GA tertinggal jauh karena alasan yang sama (Subbab 4.7.6). Durasi SA dan GA hanya naik sedikit, karena biaya per iterasi bergantung pada banyak paket di setiap truk, bukan pada banyak truk.

4.7 Analisis

Tabel 4.9: Perbandingan konfigurasi terbaik tiap algoritma (rata-rata 3 *run*)

Instans	Algoritma	Akhir	Gap (%)	Durasi (s)
small	HC *steepest*	680,7	7,3	0,29
	HC *random-restart*	712,7	2,9	1,21
	SA (20.000 iterasi)	**734,0**	**0,0**	4,63
	GA (pop. 50, gen. 200)	698,3	4,9	1,00
medium	HC *steepest*	2213,7	7,1	53,51
	HC *random-restart*	2260,7	5,2	120,72
	SA (20.000 iterasi, $p_0 = 0{,}01$)	2233,3	6,3	23,04
	SA (40.000 iterasi, $p_0 = 0{,}8$)	**2262,3**	**5,1**	37,29
	GA (pop. 50, gen. 200)	1477,7	38,0	0,67
large	HC *steepest*	5205,7	7,6	693,51
	HC *random-restart*	**5261,7**	**6,6**	1683,70
	SA (20.000 iterasi, *ratio bias* 0,2)	5076,3	9,9	21,30
	GA (pop. 100, gen. 100)	2265,7	59,8	10,70

4.7.1 Seberapa dekat dengan optimum global?

Pada instans *small*, SA mencapai 734 pada ketiga *run*. Nilai ini sama dengan batas atas, sehingga hasil SA *terbukti* merupakan optimum global. HC *random-restart* tertinggal 2,9% dan GA 4,9%. Pada *medium* dan *large*, HC dan SA berada 5–10% di bawah batas atas (HC *stochastic* 12–18%). Angka ini merupakan batas atas jarak, karena batas atas *knapsack* mengabaikan geometri: kombinasi paket yang muat berdasarkan berat dan volume belum tentu dapat disusun tanpa celah, tanpa melayang, dan tanpa bertumpu pada paket *fragile*. Jarak sebenarnya ke optimum global kemungkinan lebih kecil.

GA tertinggal jauh pada *medium* (38%) dan *large* (60%). Penyebabnya dibahas pada Subbab 4.7.6.

4.7.2 Perbandingan antaralgoritma

- **HC *steepest-ascent*** cepat naik pada awal pencarian (Gambar 4.1), tetapi berhenti di *local optimum* setelah 7–65 iterasi. Hasilnya sudah cukup baik karena setiap langkah memilih yang terbaik dari ribuan tetangga.
- **HC *stochastic*** hasilnya 0,5–11% di bawah *steepest* (gap 8–18%) dengan waktu lebih singkat (0,1 detik di *small*, 19 detik di *medium*, dan 10 menit di *large*). Langkahnya acak di antara tetangga yang lebih baik, bukan yang terbaik, sehingga setiap langkah naik lebih sedikit dan puncak yang dicapai umumnya lebih rendah. Pada *large*, dua dari tiga *run* berhenti karena `max_iterations`=100, sehingga hasilnya masih dapat naik dengan iterasi lebih banyak.

- **HC *sideways move*** tidak lebih baik daripada *steepest*. Semua *run* berhenti karena batas *sideways move*. Artinya banyak *plateau* (misalnya rotasi atau *swap* paket di luar truk yang tidak mengubah nilai), tetapi berjalan di *plateau* jarang menemukan jalan naik.
- **HC *random-restart*** adalah varian HC terbaik dan paling konsisten, dengan biaya waktu beberapa kali lipat *steepest* karena setiap *restart* adalah satu pendakian penuh.
- **SA** terbaik di *small*, setara HC *random-restart* di *medium* bila diberi 40.000 iterasi, dan sedikit di bawahnya di *large*. Keunggulannya adalah waktu: puluhan detik dibanding puluhan menit HC pada *large*.
- **GA** terburuk pada instans sedang dan besar meskipun waktunya singkat.

4.7.3 Perbandingan durasi

Biaya HC *steepest* dan *sideways* per iterasi sebanding dengan banyak tetangga, yaitu $O(n^2)$ *swap* ditambah $O(n^2)$ *move* ke *extreme point*. Setiap tetangga juga di-*repair*, yang biayanya kuadratik terhadap banyak paket di truk. Karena itu waktunya tumbuh sangat cepat: sekitar 0,3 detik di *small*, 50 detik di *medium*, dan 10–16 menit di *large*. *Random-restart* sekitar 2–2,5 kali *steepest*. HC *stochastic* hanya memeriksa seluruh tetangga ketika 50 tetangga acak tidak memberi perbaikan. Pada satu *run* *medium*, hal ini terjadi pada 16 dari 36 iterasi dan menghabiskan 98% waktunya. Karena itu durasinya lebih singkat daripada *steepest*, tetapi tumbuh dengan cara yang sama terhadap ukuran instans. Sebaliknya, SA dan GA hanya menilai satu tetangga per iterasi atau per anak, sehingga biayanya hampir linear terhadap banyak iterasi (Tabel 4.4: 2.500 iterasi 1,7 detik, 40.000 iterasi 37 detik). GA paling cepat karena total anak yang dievaluasi hanya populasi × generasi (paling banyak 10.000).

Catatan: eksperimen SA dijalankan pada waktu yang berbeda dari eksperimen HC dan GA. Pengukuran ulang satu *run* SA *medium* 20.000 iterasi pada mesin yang sama dengan HC hanya memerlukan sekitar 3 detik (Tabel 4.3 mencatat sekitar 23 detik). Karena itu, perbandingan durasi antaralgoritma dibaca sebagai orde besaran. Kesimpulan bahwa HC jauh lebih lambat daripada SA pada instans besar tidak berubah.

4.7.4 Konsistensi hasil

SA pada *small* memberikan hasil identik di ketiga *run*. Pada *medium* dan *large*, HC *random-restart* paling konsisten (simpangan baku 7 dan 46), disusul SA (21 dan 42–75) dan HC *steepest*. HC *stochastic* kurang konsisten (simpangan baku 54–123) karena langkahnya dipilih secara acak, dan GA paling tidak konsisten. Simpangan baku GA mencapai 245 pada *large*, karena hasilnya sangat bergantung pada individu awal terbaik yang kebetulan terbentuk.

4.7.5 Pengaruh parameter Simulated Annealing

- **Banyak iterasi** adalah parameter paling berpengaruh: nilai akhir naik dari 1985,7 (2.500 iterasi) menjadi 2262,3 (40.000 iterasi). Frekuensi *stuck* ikut naik, karena dengan jadwal yang lebih panjang lebih banyak waktu dihabiskan pada suhu rendah.
- p_0 **kecil (0,01)** memberi hasil terbaik pada 10.000 iterasi. Dengan p_0 besar, sebagian besar iterasi awal terbuang untuk menerima langkah buruk. Karena setiap *state* sudah di-*repair* menjadi valid, sehingga eksplorasi berlebihan tidak diperlukan.
- **Jadwal geometrik** jauh lebih baik daripada linear (2187,7 vs 1755,7). Jadwal linear mempertahankan suhu tinggi terlalu lama dan baru dingin di akhir, sehingga waktu untuk memperbaiki solusi sangat sedikit.
- $T_{\min}$ **kecil** lebih baik. Dengan $T_{\min} = 5$, langkah memburuk masih sering diterima di akhir, sehingga solusi tidak sempat "mengeras".
- ***Ratio bias*** sedikit menaikkan rata-rata, yaitu +0,6% di *medium* dan +2,0% di *large* untuk bias 0,2.

4.7.6 Pengaruh populasi dan generasi pada Genetic Algorithm

Pada *small* dan *medium*, menambah generasi (50 → 200) dan menambah populasi (20 → 100) sama-sama menaikkan nilai akhir. Pada *medium* nilainya naik dari 1221,7 ke 1477,7 untuk generasi dan dari 1220,0 ke 1422,7 untuk populasi. Kurva pada Gambar 4.5 masih naik di akhir, sehingga GA belum konvergen. Durasi naik linear terhadap populasi × generasi. Pada *large*, pengaruh keduanya hampir tertutup oleh variansi: banyak *run* tidak pernah melampaui individu terbaik generasi ke-0 (kolom "Gen. terbaik" bernilai 0 pada Tabel A.1).

Rendahnya kinerja GA disebabkan oleh operator *crossover*. *Single-point crossover* menggabungkan koordinat paket dari dua susunan yang tidak berhubungan. Posisi yang baik pada satu induk hanya bermakna relatif terhadap paket lain di susunan induk itu, sehingga anak umumnya berisi banyak paket yang bertumpuk. *Repair* kemudian mengeluarkan paket-paket tersebut. Pengukuran tambahan pada 200 pasangan induk acak dari populasi awal menunjukkan bahwa anak (setelah *repair*) lebih baik daripada induk terbaiknya hanya pada 8% (*small*), 6% (*medium*), dan 2% (*large*) kasus. Perbaikan nilai praktis hanya berasal dari mutasi, yaitu satu langkah per anak dengan peluang 0,2. Jumlah ini jauh lebih sedikit daripada puluhan ribu langkah SA. Selain itu, populasi awal acak setelah *repair* hanya berisi sekitar separuh paket yang semula dimasukkan, sehingga titik awal GA rendah. Representasi yang cocok untuk *crossover* versi salindia (*string* dengan gen yang independen, seperti 8-*queens*) tidak cocok untuk persoalan *packing* yang gennya saling bergantung secara geometris.

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

1. Ketiga algoritma *local search* (Hill-Climbing dengan keempat variannya, Simulated Annealing, dan Genetic Algorithm) berhasil diimplementasikan untuk persoalan pengisian peti kemas truk. Seluruh solusi akhir memenuhi batasan dinding, tumpang tindih, tumpuan, *fragile*, dan kapasitas berkat heuristik *repair*.

2. Simulated Annealing memberikan keseimbangan terbaik antara kualitas dan waktu. SA mencapai optimum global (terbukti oleh batas atas *knapsack*) pada instans *small* di ketiga *run*, dan berada 5–10% dari batas atas pada instans yang lebih besar dalam hitungan detik.

3. HC *random-restart* memberikan hasil terbaik dan paling konsisten pada instans *medium* dan *large*, tetapi waktunya paling lama (sekitar 28 menit per *run* pada *large*) karena membangkitkan seluruh tetangga di setiap iterasi. HC *stochastic* lebih cepat daripada *steepest*, tetapi hasilnya lebih rendah karena memilih tetangga yang lebih baik secara acak, bukan yang terbaik.

4. Genetic Algorithm versi salindia (*roulette wheel*, *single-point crossover*, mutasi) bekerja cukup baik pada instans kecil, tetapi tertinggal jauh pada instans sedang dan besar. *Crossover* yang menukar ekor kromosom merusak susunan geometris yang dibangun induk. Menambah populasi dan generasi tetap meningkatkan hasil, tetapi tidak menutup selisih tersebut.

5. Parameter terpenting SA adalah banyak iterasi dan jadwal suhu. Jadwal geometrik dengan p_0 dan $T_{\min}$ kecil memberikan hasil terbaik.

6. Objektif alternatif berbasis ETA berhasil menggeser pilihan paket ke paket yang lebih mendesak dengan pengorbanan total *value* yang kecil (3,8%).

7. Dukungan multi-truk diperoleh hanya dengan menambahkan identitas truk pada *state* serta memeriksa batasan per truk, tanpa mengubah ketiga algoritma. Dengan dua truk, SA dan HC *steepest* mengangkut 14–19% lebih banyak *value* dan berada 2–4% dari batas atas.

5.2 Saran

1. **Representasi GA berbasis urutan.** Kromosom dapat diubah menjadi urutan paket beserta orientasinya, lalu didekode secara *greedy* ke *extreme point* pertama yang valid. Dengan *order crossover*, anak selalu berupa susunan valid yang mewarisi urutan prioritas induknya.

2. **Inisialisasi dan pengisian *greedy*.** Uji coba awal di luar kode utama, yaitu menambahkan langkah pengisian *greedy* ke *extreme point* setelah *repair*, menaikkan rata-rata GA (populasi 50, generasi 100) dari sekitar 1439 menjadi 2313 pada *medium* dan dari 2302 menjadi 5370 pada *large*. Sebagian besar kenaikan berasal dari heuristik *greedy* itu sendiri, sehingga pendekatan ini perlu dikaji sebagai algoritma memetik.

3. **Evaluasi inkremental.** *Repair* dan pemeriksaan batasan saat ini menghitung ulang seluruh *state* untuk setiap tetangga. Evaluasi inkremental (hanya memeriksa paket yang berubah) akan mempercepat HC *steepest* secara signifikan.

4. **Eksekusi paralel.** *Run* antar-*seed* dan antarkonfigurasi independen, sehingga eksperimen dapat dijalankan paralel. Pengukuran durasi sebaiknya tetap dilakukan dalam kondisi yang sama untuk semua algoritma.

Bab 6

Pembagian Tugas

Tabel 6.1: Pembagian tugas anggota kelompok

Nama	**NIM**	**Tugas**
Athilla Zaidan Zidna Fann	13524068	Genetic Algorithm
Jason Edward Salim	13524034	Simulated Annealing
Raynard Fausta	13524052	Hill-Climbing, multi-truk (bonus)

Referensi

[1] Asisten IF3170, "Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027," Edunex ITB, 2026.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hall, 2010.

[3] M. L. Khodra, "Beyond classical search: Classical vs local search," YouTube [Video], 2021, diakses 16 September 2026. [Online]. Available: https://www.youtube.com/watch?v=oGTa9kGE3P8

[4] M. L. Khodra, "Beyond classical search: Hill-climbing search," YouTube [Video], 2021, diakses 16 September 2026. [Online]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8

[5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[6] M. L. Khodra, "Beyond classical search: Simulated annealing," YouTube [Video], 2021, diakses 16 September 2026. [Online]. Available: https://www.youtube.com/watch?v=AdhG7HnhYrk

[7] M. L. Khodra, "Beyond classical search: Genetic algorithm," YouTube [Video], 2021, diakses 16 September 2026. [Online]. Available: https://www.youtube.com/watch?v=aI41TGo9J9c

[8] KK Informatika STEI ITB, "Salindia IF3170 Inteligensi Artifisial, Modul 3: Beyond Classical Search (Hill-Climbing, Simulated Annealing, Genetic Algorithm)," Edunex ITB, 2026.

[9] T. G. Crainic, G. Perboli, and R. Tadei, "Extreme point-based heuristics for three-dimensional bin packing," *INFORMS Journal on Computing*, vol. 20, no. 3, pp. 368–384, 2008.

[10] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, 1990.

[11] W. Ben-Ameur, "Computing the initial temperature of simulated annealing," *Computational Optimization and Applications*, vol. 29, no. 3, pp. 369–385, 2004.

Lampiran A

Lampiran

A.1 Tautan Repository

`https://github.com/AthillaZaidan/IF3170_G04_BUMC`

A.2 Hasil Setiap Run Genetic Algorithm

Tabel A.1: Hasil setiap *run* Genetic Algorithm

Pop.	Gen.	Seed	Awal	Akhir	Durasi (s)	Gen. terbaik
small						
50	50	1	470	563	0.10	15
50	50	2	477	617	0.19	27
50	50	3	518	644	0.34	45
50	100	1	486	653	0.34	39
50	100	2	506	629	0.44	77
50	100	3	466	640	0.37	67
50	200	1	343	708	0.87	120
50	200	2	379	679	1.13	111
50	200	3	420	708	1.09	99
20	100	1	396	548	0.23	35
20	100	2	382	599	0.21	70
20	100	3	389	561	0.16	87
100	100	1	446	655	1.20	55
100	100	2	410	667	1.00	68
100	100	3	455	644	1.35	41
medium						
50	50	1	980	1190	0.15	44
50	50	2	1289	1289	0.16	0
50	50	3	978	1186	0.14	29
50	100	1	970	1246	0.34	91
50	100	2	1015	1558	0.36	88
50	100	3	991	1154	0.31	92
50	200	1	983	1485	0.64	192
50	200	2	1115	1369	0.69	166
50	200	3	1006	1579	0.67	198
20	100	1	947	1120	0.14	97
20	100	2	1047	1316	0.12	82
20	100	3	931	1224	0.15	90
100	100	1	964	1508	0.67	94
100	100	2	1029	1407	0.61	99
100	100	3	987	1353	0.60	81
large						

Pop.	Gen.	Seed	Awal	Akhir	Durasi (s)	Gen. terbaik
50	50	1	2099	2099	1.02	0
50	50	2	2001	2077	0.78	43
50	50	3	1990	2064	1.15	39
50	100	1	1932	1932	2.27	0
50	100	2	2273	2394	2.47	42
50	100	3	2019	2019	2.52	0
50	200	1	2014	2021	6.55	200
50	200	2	2090	2345	6.77	172
50	200	3	1847	1923	5.80	189
20	100	1	2092	2163	1.56	100
20	100	2	1978	2130	1.18	95
20	100	3	2140	2340	1.13	79
100	100	1	2071	2071	8.09	0
100	100	2	2307	2458	12.91	81
100	100	3	2165	2268	11.07	90

A.3 Pernyataan Penggunaan AI Generatif

Kelompok menggunakan asisten AI generatif selama pengerjaan tugas ini. Seluruh keluaran AI telah diperiksa, dijalankan, dan dipahami oleh anggota kelompok.

Tabel A.2: Penggunaan AI generatif

Alat	Penggunaan
Claude	Mempelajari secara lebih mendalam cara kerja ketiga algoritma (Hill-Climbing, Simulated Annealing, dan Genetic Algorithm).